

Exercise: Stored Procedures and Transactions

Objective

Create, modify, and remove stored procedures to demonstrate your understanding of the concepts covered in the lesson.

Task 1: Create a Simple Stored Procedure

1. Create a stored procedure in PostgreSQL that selects all the columns from a table named Customers where the Status is 'Active'.
2. Call your stored procedure to verify that it works as expected.

PostgreSQL

```
CREATE PROCEDURE GetActiveCustomers()  
LANGUAGE SQL  
AS $$  
    SELECT * FROM Customers  
    WHERE Status = 'Active';  
$$;
```

```
CALL GetActiveCustomers();
```

Task 2: Create a Stored Procedure with a Single Parameter

1. Create a stored procedure that accepts an INT parameter for CustomerId and returns the customer's details from the Customers table.
2. Call your stored procedure with a specific CustomerId to retrieve their information.

PostgreSQL

```
CREATE PROCEDURE GetCustomerDetails(IN CustomerIdIn INT)  
LANGUAGE SQL  
AS $$  
    SELECT * FROM Customers  
    WHERE CustomerId = CustomerIdIn;  
$$;
```

```
CALL GetCustomerDetails(1);
```

Task 3: Create a Stored Procedure with Multiple Parameters

1. Create a stored procedure that takes two parameters: an INT for CustomerId and a VARCHAR for CustomerName, then returns customer details where both CustomerId and CustomerName match.
2. Call your stored procedure with specific values for CustomerId and CustomerName.

PostgreSQL

```
CREATE PROCEDURE GetCustomerInfo(IN CustomerIdIn INT, IN CustomerNameIn VARCHAR(255))
LANGUAGE SQL
AS $$
    SELECT * FROM Customers
    WHERE CustomerId = CustomerIdIn AND CustomerName = CustomerNameIn;
$$;
```

```
CALL GetCustomerInfo(1, 'John Doe');
```

Task 4: Modify a Stored Procedure

1. Modify the stored procedure from Task 2 to include a check that only returns the customer's details if they are active (Status = 'Active').
2. Call your modified stored procedure to verify the update.

PostgreSQL

```
DROP PROCEDURE IF EXISTS GetCustomerDetails;
CREATE PROCEDURE GetCustomerDetails(IN CustomerIdIn INT)
LANGUAGE SQL
AS $$
    SELECT * FROM Customers
    WHERE CustomerId = CustomerIdIn AND Status = 'Active';
$$;
```

```
CALL GetCustomerDetails(1);
```

Task 5: Remove a Stored Procedure

1. Remove the stored procedure created in Task 3 from the database.

PostgreSQL

```
DROP PROCEDURE IF EXISTS GetCustomerInfo;
```

Reflection Questions

1. Why are stored procedures useful in maintaining consistency across database operations?
2. How can stored procedures improve performance and security in database systems?
3. What are the key differences between writing stored procedures in PostgreSQL and MySQL?

This exercise will help you solidify your understanding of stored procedures, their creation, modification, and removal in a PostgreSQL environment.